

Documentazione: Sistema di Gestione Ospedaliera (SecondoHomework)

Componenti del Gruppo: Iazzetta Angelo Karol, Di Domenico Lorenzo, Mottola Giuseppe
Anno Accademico: 2025/2026

1. Introduzione

Il presente documento descrive l'architettura software del progetto "Sistema Gestione Ospedaliera". L'obiettivo del sistema informativo è fornire un applicativo Java dotato di interfaccia grafica (GUI) in Swing e supportato da una base di dati relazionale, in grado di gestire in modo efficiente le risorse strutturali, i ricoveri dei pazienti e i turni del personale medico.

L'applicazione è strutturata secondo il pattern architetturale MVC (Model-View-Controller) per garantire una separazione tra la logica di dominio, il flusso applicativo e l'interfaccia utente. Il sistema pone particolare attenzione sulla coerenza dei dati, impedendo sovrapposizioni temporali nell'allocazione dei letti e nella programmazione delle prestazioni mediche. Inoltre, essendo il progetto sviluppato da un gruppo di tre persone, il sistema implementa la gestione avanzata dello stato di "Malattia" del personale, automatizzando la ricerca di medici sostitutivi per i turni scoperti.

2. Architettura dei Package

Il codice sorgente è stato suddiviso in tre package per rispettare i principi di modularità e coesione:

- `model`: Contiene le entità del dominio e la logica di business fondamentale.
- `controller`: Gestisce il flusso dell'applicazione, intercetta le azioni dell'utente e gestisce la comunicazione tra l'interfaccia e il modello.
- `gui`: Contiene le classi dedicate all'Interfaccia Grafica Utente (Graphical User Interface).

3. Il Package `model`

Il package `model` rappresenta il cuore logico dell'applicazione, in cui sono state implementate le classi che mappano le entità reali dell'ospedale, avvalendosi dei concetti di ereditarietà e polimorfismo.

3.1 Gestione delle Utenze e dei Ruoli

`Utente`: Classe base che incapsula i concetti fondamentali di autenticazione (login e password) necessari per l'accesso univoco al sistema.

`Amministratore`: Sottoclasse di `Utente` che gestisce il personale amministrativo. Ha i privilegi per gestire le anagrafiche, allocare i ricoveri nei letti, gestire le assenze per malattia e visualizzare le statistiche dei pazienti in dimissione.

Medico: Sottoclasse di `Utente` che rappresenta il personale sanitario. È associato in modo esclusivo a un singolo `Reparto`. Possiede liste di `TurnoLavorativo` e `Prestazione`, gestite in modo da evitare accavallamenti temporali.

Paziente: Gestisce l'anagrafica del paziente ospedaliero, entità fondamentale per la registrazione dei ricoveri.

3.2 Gestione Strutturale

L'infrastruttura fisica dell'ospedale è modellata in modo gerarchico:

Reparto: Entità macroscopica che raggruppa logicamente le stanze e il personale medico afferente.

Stanza: Contenitore spaziale per i letti.

Letto: Identifica la singola postazione univoca all'interno dell'intero ospedale.

3.3 Processi Clinici e Organizzativi

Ricovero: Associa un paziente a un letto specifico per un intervallo di tempo (data/ora di inizio e dimissione).

TurnoLavorativo: Definisce le fasce orarie programmate in cui un medico presta servizio nel proprio reparto.

Prestazione: Classe astratta che rappresenta un'attività clinica erogata da un medico durante un ricovero. Viene specializzata in due sottoclassi concrete per differenziare i dati necessari:

Visita: Contiene attributi specifici come il tipo `Visita`.

Intervento: Contiene dati strutturali aggiuntivi come il numero di `salaOperatoria`.

Malattia: Definisce il periodo di assenza di un medico e permette all'amministratore di eseguire la logica di ricerca per le sostituzioni.

4. Il Package controller

Il package controller viene implementato attraverso il `Controller`, il quale agisce come gestore dello stato applicativo (mantenendo in memoria le liste di tutte le entità a runtime) e come garante assoluto dell'integrità dei dati.

Le responsabilità della classe `Controller` si dividono in diverse aree funzionali che sono progettate per soddisfare i vincoli della traccia:

- **Autenticazione e Inizializzazione:** Attraverso il metodo `effettuaLogin`, il controller intercetta gli inserimenti dalla GUI e verifica la validità delle credenziali ciclando la lista degli utenti registrati.
- **Gestione Ricoveri e Prevenzione Sovrapposizioni:** Il sistema implementa una logica di validazione temporale per l'assegnazione delle risorse. Il metodo `aggiungiRicovero` si avvale di una funzione privata di supporto (`overlap`) per calcolare eventuali intersezioni di date. Questo impedisce categoricamente l'immatricolazione di un nuovo paziente su un `Letto` che risulta già occupato in un intervallo di tempo coincidente. La chiusura formale dell'assegnazione avviene poi tramite il metodo `aggiungiDimissione`.

- **Validazione delle Prestazioni Mediche:** Anche i metodi operativi `registraVisita` e `registraIntervento` incorporano controlli incrociati. Prima di confermare l'inserimento, il controller effettua due verifiche:
 1. Si assicura che l'orario della prestazione ricada all'interno di un `TurnoLavorativo` valido e assegnato a quel medico.
 2. Applica la funzione di overlap sull'agenda del medico per impedire l'accavallamento con prestazioni da lui stesso già prese in carico.
- **Motore di Ricerca Sostituzioni:** Il metodo `suggerisciSostituti` identifica il medico assente e il sistema isola i colleghi appartenenti al suo stesso Reparto. Successivamente, filtra questa lista escludendo in automatico qualsiasi medico la cui agenda presenti già un turno o una prestazione in conflitto con l'intervallo di assenza, restituendo alla GUI un elenco pulito di candidati idonei alla sostituzione.
- **Estrazione Dati e Viste:** Il controller utilizza metodi mirati per popolare in modo dinamico le interfacce grafiche, fungendo da strato di interrogazione. Esempi chiave sono `agendaGiornaliera` e `agendaSettimanale` per la dashboard del medico, `cercaLettiDisponibili` (che esclude dinamicamente i letti occupati) e `dimissioniInData` per le attività di monitoraggio dell'amministratore.

5. Il Package gui

Il package `gui` racchiude l'interfaccia visiva con la quale l'utente interagisce, sviluppata utilizzando la libreria standard Java Swing. Le interfacce sono state strutturate per offrire un'esperienza utente chiara e guidata.

5.1 Finestre Principali e Autenticazione

- **login:** Rappresenta la schermata di avvio del sistema. Raccoglie le credenziali (nome utente e password) e richiede la validazione al controller. In base al polimorfismo dell'oggetto `Utente` restituito (se `Amministratore` o `Medico`), questa classe chiude la finestra corrente e istanzia e rende visibile la dashboard appropriata.

5.2 Dashboard e Schermate Operative

Il sistema si divide in interfacce specifiche a seconda del ruolo dell'utente loggato:

- **Amministratore (Dashboard Amministratore):** È il pannello di controllo centrale per il personale amministrativo. Da questa finestra è possibile:
 - **Gestione Pazienti e Ricoveri:** Inserire nuovi pazienti nell'anagrafica ed effettuare accettazioni registrando nuovi ricoveri (associando matricola paziente e matricola letto).
 - **Dimissioni:** Registrare la fine di un ricovero e consultare, tramite l'apposita lista, tutte le dimissioni previste o effettuate filtrate per una specifica data.
 - **Gestione Malattie:** Tramite un'apposita finestra di dialogo, l'amministratore può assegnare un periodo di assenza per malattia a un medico, innescando così nel controller la ricerca dei sostituti.

- **GestioneLetto:** È una finestra di utilità richiamabile dalla dashboard dell'amministratore. Consente la ricerca rapida della disponibilità dei posti letto in un dato reparto. Per soddisfare i vincoli di traccia, la lista implementa un DocumentListener per la ricerca in tempo reale e un ListCellRenderer personalizzato che formatta testualmente i letti, evidenziando automaticamente in **rosso** i letti attualmente occupati e in **verde** quelli disponibili.
- **Medico (Dashboard Medico):** È l'ambiente di lavoro dedicato al personale sanitario. L'interfaccia è divisa in due aree logiche:
 - **Registrazione Prestazioni:** Permette al medico di refertare le prestazioni effettuate sui pazienti ricoverati. I pulsanti separati ("Registra visita" e "Registra intervento") istanziano gli oggetti corretti, richiedendo campi specifici (come il tipo di visita o il numero della sala operatoria) ed evidenziando eventuali sovrapposizioni segnalate dal controller. È inoltre presente un pulsante per aggiornare unicamente l'esito di una prestazione esistente.
 - **Agenda:** Tramite due appositi pulsanti ("Agenda giornaliera" e "Agenda settimanale"), il medico può interrogare il sistema passando una data di riferimento. Il risultato viene caricato e formattato dinamicamente in una JList, mostrando cronologicamente tutti gli impegni (visite e interventi) a lui assegnati.

link repository github: <https://github.com/OOP2026/Gruppo12/tree/main>